DALIL TRAINING ACADEMY · FULL STACK WEB DEVELOPMENT

Phase 0: Reading Guide and Tasks

Expense Tracker · Days 1 – 2 · Self-study

Before you start

- **Goal:** learn 5 new topics, and check that you understood them with 3 small tasks.
- **The backend is built right after this phase (Phase 1, Day 3).** So you need to finish these topics well before you start, because there will be no time to go back to them once you are building.
- **Read and try:** every time you read an example, open VS Code or the Console and run it. Reading without trying does not stay in your memory.
- The resources are in English and written simply. You do not need to understand every word. Focus on the examples and the code.
- Do not read all the side links on the pages. Read only the part you need.
- The times are estimates. If you need more, that is fine, but **if you are stuck for more than 30 minutes, ask in Discord.**

Plan of the days

Day	Reading	Tasks
1	Environment setup (from the Roadmap), then topics 1, 2, and 3	Task 1 (fetch + cards)
2	Topics 4 and 5	Task 2 (Express) and task 3 (Express + PostgreSQL)

Topics and resources

Topic 1: JSON

Resource	What to read	Time (est.)
MDN: Working with JSON `https://developer.mozilla.org/docs/Learn_web_development/Core/Scripting/JSON`	What JSON is, how it is different from a JavaScript object, and `JSON.parse` and `JSON.stringify`. This is the format the front-end and backend use to talk to each other for the whole project.	25 min

After reading, you should be able to answer

- What is the difference between JSON and a JavaScript object?
- When do we use `JSON.stringify` and when do we use `JSON.parse`?

Topic 2: Promises, async/await, and fetch

Resource	What to read	Time (est.)
javascript.info: Async/await https://javascript.info/async-await	What `async` and `await` mean, and how to handle errors with `try/catch`.	30 min
MDN: Making network requests with JavaScript https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Network_requests	How to use `fetch` to get data from a server and show it on the page.	40 min
MDN: Using Fetch https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch	**Reference:** come back to it when you need to send a POST with a body and headers. You do not need to read all of it.	When needed

After reading, you should be able to answer

- Why do we need `await` twice when we call `fetch` and read the JSON?
- What happens if the server answers with 404? Does `fetch` throw an error by itself? (search for `response.ok`)

Topic 3: The backend and REST

Resource	What to read	Time (est.)
MDN: Client-server overview https://developer.mozilla.org/en-US/docs/Learn/Server-side/First_steps/Client-Server_overview	The part that explains the HTTP request and response, and the methods like GET and POST.	30 min
MDN: HTTP response status codes https://developer.mozilla.org/en-US/docs/Web/HTTP/Status	**Reference:** focus on 5 codes only: 200, 201, 400, 404, and 500.	15 min

After reading, you should be able to answer

- What is the difference between GET, POST, PUT, and DELETE? What do we use each one for?
- Where do we send the data in a POST, and which status code do we return when we add something new?

Topic 4: Node.js, Express, and CORS

Resource	What to read	Time (est.)
Node.js: Introduction https://nodejs.org/en/learn/getting-started/introduction-to-nodejs	What Node.js is, and why we can run JavaScript outside the browser. Read it quickly.	15 min

Resource	What to read	Time (est.)
Express: Hello world example `https://expressjs.com/en/starter/hello-world.html`	Build and run your first server yourself, step by step.	30 min
MDN: Cross-Origin Resource Sharing (CORS) `https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS`	The introduction only: what it is and why the browser blocks some requests. You do not need the details of preflight.	20 min

After reading, you should be able to answer

- What is the difference between running JavaScript in the browser and in Node.js?
- Why must we stop and start the server again after we change its code?
- Why can the browser stop the front-end from talking to a server on another port?

Topic 5: Node.js and PostgreSQL

Resource	What to read	Time (est.)
node-postgres: Queries `https://node-postgres.com/features/queries`	Queries with parameters (`$1` and `$2`), and how to reach the result with `result.rows`. Read the warning about joining texts into SQL carefully.	25 min
node-postgres: Pool `https://node-postgres.com/apis/pool`	What a Pool is, why we use it, and `pool.query`. You do not need to read all the settings.	20 min

Before you start: `.env` is a text file that holds secret settings (like the database password), separated from the code. The server reads it with the `dotenv` package. We never upload it or submit it.

After reading, you should be able to answer

- Why must we send values as parameters (`$1`) instead of joining them into the SQL text?
- Where do we find the rows that `pool.query` returned?
- Why is the database password written in the `.env` file, not in the code?

The small tasks

The tasks are small versions of what you will build in the project. Solve them yourself. If you get stuck, go back to the resources above before you ask.

Task 1: fetch and Bootstrap cards — Day 1

Goal: get real data from an API and show it on the page.

Steps

- Build an HTML page with Bootstrap (CDN).
- Write an `async` function that gets `https://jsonplaceholder.typicode.com/users` with `fetch`.
- For every user, show a Bootstrap card with: the name, the email, the city (`address.city`), and the company name (`company.name`).
- The cards are built with the DOM from the data, not written by hand in the HTML.
- Use `try/catch` and show a Bootstrap alert if the request fails. Try a wrong URL on purpose to see the alert.
- Show a Bootstrap spinner while loading.

I am done when…

- ☐ 10 cards with the right data appear.
- ☐ When I break the URL, an alert appears, not an empty page.
- ☐ I can explain why I used `await` twice.

Task 2: Your first Express server — Day 2

Goal: build a small server that returns JSON, and see with your own eyes why we need CORS.

Steps

- Create a new folder and run `npm init -y`, then `npm install express`.
- Create a `server.js` file with a route on `GET /api/hello` that returns JSON with a message. Use `res.json`.
- Add a second route `GET /api/expenses` that returns an array with two fixed expenses, in the same JSON shape as the Roadmap.
- Run the server with `node server.js`, open the link in the browser, then try it from Thunder Client.
- Try a link that does not exist and see what it returns.
- **CORS by experiment:** make a small HTML page and open it with Live Server, and `fetch` from `http://localhost:3000/api/expenses`. You will see a CORS error in the Console.
- Then fix it: run `npm install cors`, add the line `app.use(cors())` to the server, and try again.

I am done when…

- ☐ Both routes return JSON from the browser and from Thunder Client.
- ☐ I saw a CORS error with my own eyes, and I know the fix.
- ☐ I can explain in my own words what status code `404` is.

Task 3: Express and PostgreSQL — Day 2

Goal: connect an Express server to PostgreSQL and read from it, and see what the `pg` library really returns.

Steps

- Use the folder of task 2, and run `npm install pg dotenv`.
- Create a database named `expense_tracker_practice` in pgAdmin, and make a small table in it (for example a `students` table with two columns: `id` and `name`), and add two rows.
- Copy `.env.example` (you will find it in the starter) to a new file named `.env` and write your PostgreSQL password.
- In the server, write `require("dotenv").config()`, and create a `Pool` from the `pg` library with the values of `.env`.
- Change `GET /api/hello` (or add a new route) to return the rows from the table: `pool.query("SELECT * FROM students ORDER BY id")` and then `result.rows`. The function must be `async`.
- Add a route with a parameter: `pool.query("SELECT * FROM students WHERE id = $1", [id])`. If you do not find a row, return 404.
- Try the link with a text instead of a number for the id (like `/abc`). What happened? PostgreSQL does not accept text where a number is expected. Fix it by checking the id before the query and returning 404.

I am done when…

- ☐ The route returns the rows from the database, not from an array written in the code.
- ☐ The link with a wrong id returns 404, not a server error.
- ☐ I can explain why we used `$1` instead of writing the id inside the SQL text.

Errors you may see

These are the errors most students meet. Read the table before you ask.

Message	It usually means
`Cannot GET /something`	The route does not exist or is written wrong. Or you changed the code and did not restart the server.
`blocked by CORS policy`	The server did not allow the page to talk to it. Use the `cors` package.
`Unexpected token < ... is not valid JSON`	The answer you got was an HTML page, not JSON. The URL is usually wrong.
`Failed to fetch`	The server is not running, or the port is wrong. `ERR_CONNECTION_REFUSED` may appear with it.
`EADDRINUSE: address already in use`	Another server is running on the same port. Stop it with Ctrl+C or change the port.
`await is only valid in async functions`	You used `await` inside a function that is not `async`. Add `async` before the function.
`password authentication failed for user`	The password in the `.env` file is wrong. Check it in pgAdmin.
`database "..." does not exist`	You did not create the database, or its name in `.env` is wrong.
`relation "..." does not exist`	The table is not in the database you connected to.
`ECONNREFUSED 127.0.0.1:5432`	PostgreSQL is not running, or the port in `.env` is wrong.
`invalid input syntax for type integer`	You sent text where the database expects a number. Check the id before the query.

When you finish

Review the Phase 0 checklist in the Roadmap, and submit the folder with the three tasks at the end of Day 2. After that, you start Phase 1: building the whole backend.